

17-214/514: Agentic Software Development

Refactoring and Design Improvement

Hammad Ahmad and Claire Le Goues • Monday, September 28, 2026 • **Lecture**

Administrivia

- Midterm 1 grades have been released. Regrade requests are open until Sunday at 11:59pm.
- The Assignment 3 client is in your repository, under `ta-client/`. Pull to get it.
 - We will check that it still works with your system (the basic functionality, no weird edge cases).
 - Make sure you refresh your browser window for the assignment handout, since there are additional clarifications on there.
 - If you have any issues with your client, come talk to us.

What did we talk about in the beforetimes, pre-midterm?

- Smells are names for design problems (the classics, and a few common in generated code).
- Generated code shows the same smells for new reasons (missing context, an underspecified request, free volume).
- A smell is a signal. We fix the ones in the path of change and (optionally, if relevant for later) write down the rest.
- Today: how do we fix structure **without breaking behavior**?

Learning goals for today

- Explain what refactoring is (and is not), and keep restructuring separate from behavior change
- Pin existing behavior with characterization tests before changing anything
- Direct an agent through a bounded refactor
- Check a refactor's diff for behavior changes
- Trace what a contract change touches in a system, and decide whether to evolve the code in place or regenerate it

Make it better. Change nothing.

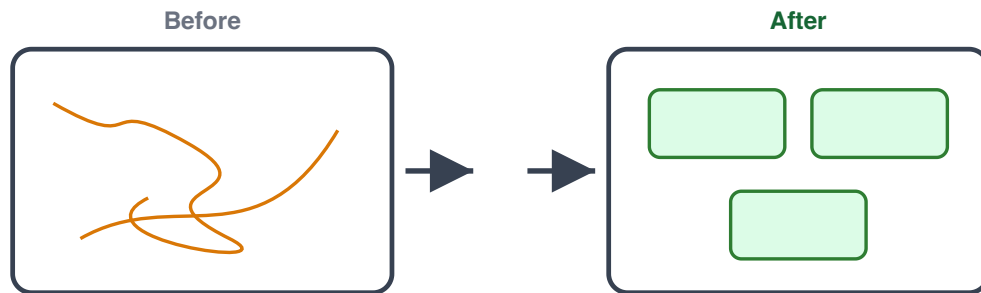
"Clean this up. But don't break anything."

- The code should improve and keep doing what it did before. Is that coherent?
- It is, if we can say what it did before and check that it still does.

Change without change

What is refactoring?

- A **refactoring** is a change to the code's internal structure that does not alter its externally observable behavior (Fowler).
- "We refactored it and now it's broken." Then it was not a refactoring. It was an optimistic edit, at best.



Same inputs, same outputs, same errors, same contract.

Refactoring is design on existing code

- Recall: design is data, operations, and structure.
- Refactoring re-chooses the structure after the code exists.
- Last time we went from "this feels off" to specific smells. Refactoring helps fix smells.
- Most code we meet was written by someone else (or generated), so we will do more of this than greenfield design.

The two hats

The restructuring hat

- The behavior is **frozen**
- You move code around
- The tests do not change
- Commit message: refactor:
...

The feature hat

- The behavior **changes**
- The tests change **first**
- Then the code follows
- Commit message: feat: ...

Wearing both hats at the same time is how "improvements" change behavior.

In practice, switch hats at a commit boundary, so the history shows which hat we wore.

"Behavior preserved" is a claim

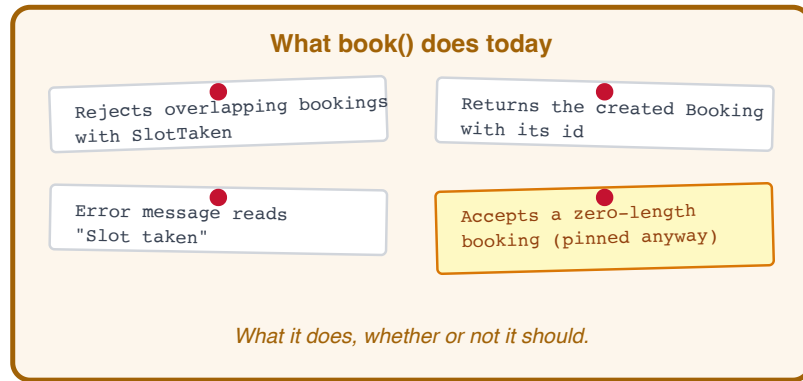
- Every refactoring makes the claim, by definition.
- The check is the suite we already have, plus some more tests.
- Agents can make this claim on every "cleanup", and it can be false.
 - In one study, GPT-4o's refactorings kept the tests passing about 45% of the time on a first try, and 90% once compile and test results were fed back (Oueslati et al.).

Oueslati, Lamothe, Khomh, **RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring**, ICSE 2026, Table 6.

The safety net

Characterization tests

- A **characterization test** records what the code does today, whether or not that is what it should do.
- If the code does something odd, the test asserts the odd thing.
- It is an alarm that rings if anything observable changes while we restructure.



Restructure freely. If a pin moves, the alarm rings.

No tests? Then no touching.

- "Legacy code" is code without tests (Feathers). Age has nothing to do with it, so three-week-old generated code qualifies.
- The trap: we cannot refactor safely without tests, and often cannot test without first opening a boundary.
- One way in: characterization tests at the outermost boundary we can reach, then work inward.



programmerhumor.io/programming-memes/i-repeat-do-not-touch-any-code-nuyp

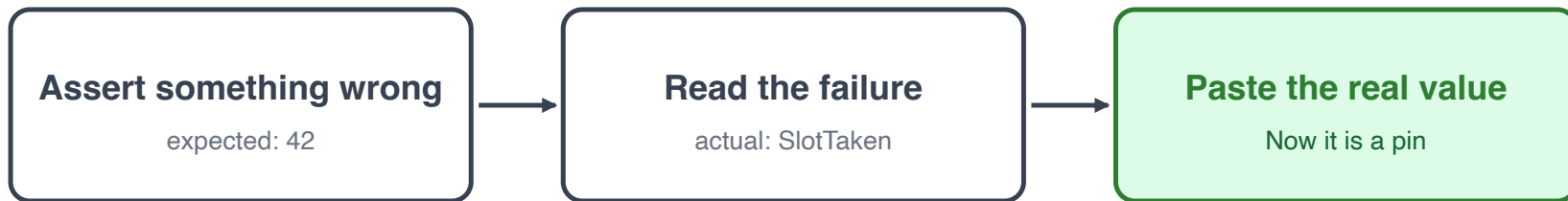
Pinning what `book()` does today

```
@Test
void pin_zeroLengthBookingIsAllowed() {
    // Not claiming this is RIGHT. Recording what the code DOES today.
    Booking b = service.book(user, room, new Slot(at(10, 0), at(10, 0)));
    assertNotNull(b);
}
```

- Our `book()` currently accepts a zero-length booking. Odd, could be a bug, but we pin it anyway.
- If the restructuring changes it, we want the alarm. (Changing it on purpose is a job for the feature hat later.)

Where does the assertion come from?

- In a normal test, the expected value comes from the spec. In a characterization test, it comes from the code.
- One way to get it (Feathers) is to assert something **wrong** on purpose, run the test, and read the real value off the failure message.
- Paste that value into the assertion. Green. The code just dictated its own test.



Generate the pins

- "Generate tests from this code" has a blind spot. The code is its own oracle.
- For characterization, the code being its own oracle is the point. We are pinning what **is**.
- So, asking the agent to write tests capturing current behavior is fine, and agents are fast at it.
- Make sure the pins make sense.
 - Ask the agent to list what each pin asserts, spot-check a few, and make one go red on purpose (for example).
 - A pin with no assertion protects nothing.
 - Check which inputs they cover, boundaries included.

When do the pins run?

- With the suite, after every move. They stay in it afterwards.
- A feature that changes pinned behavior on purpose turns the pin red. Update it. The expected value now comes from the spec, so it is an ordinary test.
- The next refactor pins whatever it touches that nothing covers yet.

For example, Jest changed an error message on purpose. Its snapshot file (a pin under another name) changed in the same pull request.

packages/jest-config/src/normalize.ts

```
    throw createConfigError(  
      - `Preset ${chalk.bold(presetPath)} not found.`,  
      + `Preset ${chalk.bold(presetPath)} not found relative to rootDir  
        ${chalk.bold(options.rootDir)}.`,  
    );
```

.../_snapshots__/_normalize.test.ts.snap

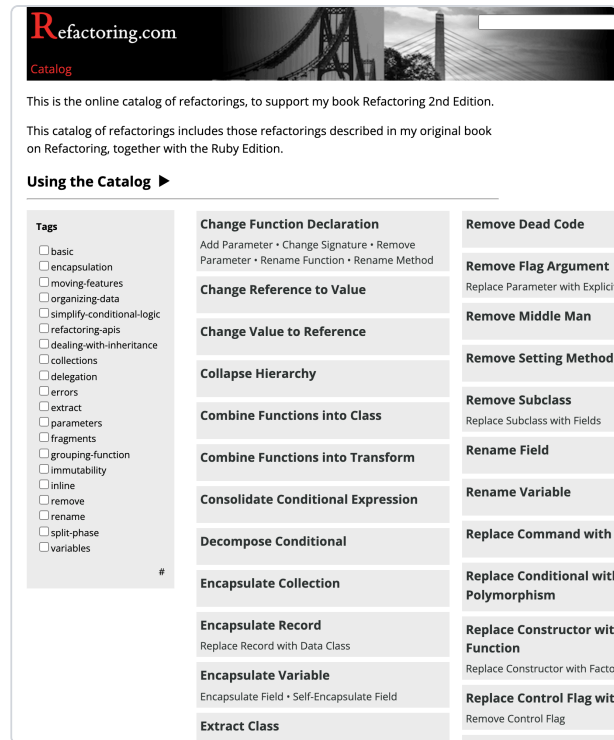
```
<red></color>  
- <red> Preset <bold>doesnt-exist</intensity> not found.</color>  
+ <red> Preset <bold>doesnt-exist</intensity> not found relative to rootDir  
  <bold>/root/path/foo</intensity>.</color>  
<red></color>
```

github.com/jestjs/jest/pull/15194

Refactoring moves

The refactoring catalog

- Refactoring comes as a catalog of small moves. Extract function, inline, move, rename, among dozens of others.
- We can request, review, and argue about a move that has a name (the same reason we named smells).
- Each move is small and reversible. A big refactor is a sequence of small moves (each one checked).



refactoring.com/catalog

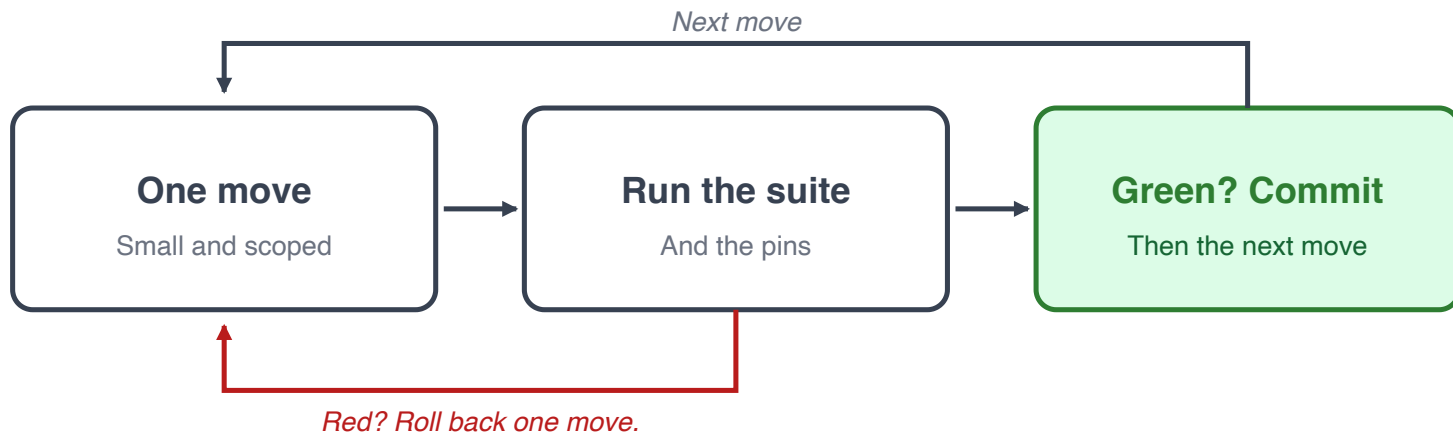
Extracting the overlap check from `book()`

```
// before: book() does the checking inline
Booking book(User u, Room r, Slot s) {
    for (Booking b : store.byRoom(r)) {
        if (b.start().isBefore(s.end()) && s.start().isBefore(b.end())
            && !b.cancelled()) {
            throw new SlotTaken();
        }
    }
    // ...
}
```

```
// after: the check has a name and a home
Booking book(User u, Room r, Slot s) {
    if (hasActiveOverlap(r, s)) throw new SlotTaken();
    // ...
}
```

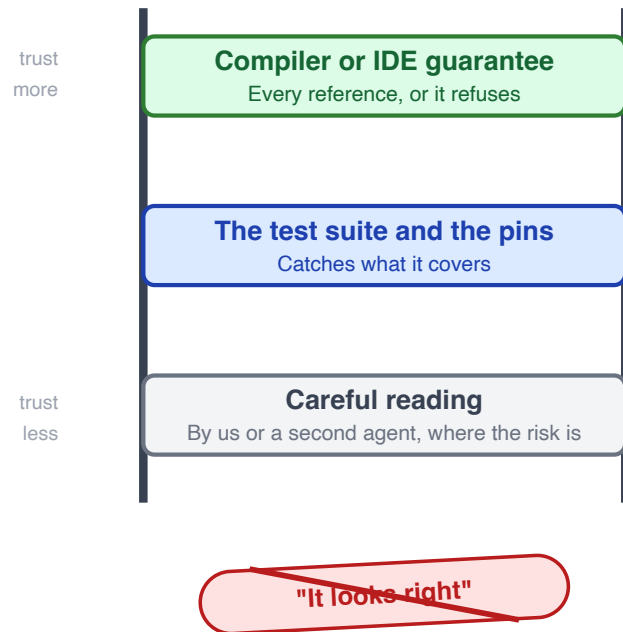
Same logic. Now the check is testable on its own, and it takes one line in `book()`. Also, it can be reused elsewhere.

The rhythm of small steps



What the IDE can promise that prose can't

- An IDE rename is backed by static analysis (and *some* agents can call the same tools). It updates every reference it can resolve, or it refuses.
- The ladder ranks kinds of check (a proof, then tests, then reading), whoever runs them. "It looks right" is not a check.
- Agents can help with reading. But, this is also where any reader, human or agent, can miss things.
 - A second agent reading the diff adversarially is worth asking for. Its findings are candidates to check.



Technical debt

The debt metaphor

- Take a structural shortcut now, and pay interest on every future change that touches it.
- The metaphor has limits. There is no statement in the mail, no interest rate, etc.
- Code that never changes again will not charge interest.
 - We can/should try to make an educated guess about which code that is (e.g., from the change history or the roadmap).

Deliberate debt and accidental mess

Deliberate debt

"We chose the simple version. Revisit when we add recurring bookings."
It is written down and dated, so we can revisit it.

Accidental mess

Nobody deliberately chose it. It built up over many generated changes. There is little or no record, so who knows what it will cost.

Agents can produce accidental mess quickly. Try to catch it when it happens. A later audit can find what we missed. At that point, we fix what is in the path of change and write down the rest.

When do we pay down the debt?

It depends.

- **Change frequency.** Debt in code we touch weekly compounds. Debt in a frozen corner is dormant.
- **Risk.** Debt next to money, security, or an invariant should get priority.
- **Cost of delay.** Some cleanups get more expensive the longer they wait (drift compounds too).

Sorting the smell inventory

- Start from the smells you wrote down and did not fix.
- Sort them by one question. Is this smell in the path of a change you expect or are implementing?
 - Yes: refactor before that change.
 - No: leave it as deliberate debt, and write it down.
 - Not sure: that one is a judgment call.

"Make the change easy, then make the easy change"

- (Kent Beck's one-sentence refactoring strategy.)
- About to add waitlists? First refactor so waitlists have an obvious home. **Then** the feature is a small diff.
- Prefer preparatory refactoring over speculative refactoring.
 - We refactor what the next change touches (and not everything).

 **Kent Beck**
@kentbeck@hachyderm.io

#MCETMEC Make the change easy, then make the easy change. When you're rushing to make a change, it's hard to take the time. Here's a great example:
[annagoodman.substack.com/p/6-i...](https://annagoodman.substack.com/p/6-i-...)

 The Second 90%
#6 I Will Alphabetize Your Case Statem...
By Anna Goodman

Feb 07, 2023, 01:41 PM · 🌐 · Web

8 boosts · 0 quotes · 16 favorites

hachyderm.io/@kentbeck/109824919832714562

When NOT to refactor

- Code that is about to be deleted. (No need to polish the doomed.)
- During a production fire. Fix first, restructure later.
- With no tests and no time to pin behavior. (Consider it gambling with extra steps.)



knowyourmeme.com/memes/lets-go-gambling-aw-dangit

Directing the refactor

Who does what in a refactor?

Agents and IDE tools: the mechanics

- Rename across forty files. Extract with every call site updated. Apply a catalog move repo-wide.
- The IDE's moves are checked by static analysis. The agent's are *mostly/usually* right.

You: which refactor

- An agent will clean up whatever you point it at, including code that will be deleted soon.
- It does not know your roadmap, your change frequency, or which corner is dormant (unless told).

How to ask for a refactor

Extract the overlap check in `BookingService.book` into a method `hasActiveOverlap(room, slot)`. Behavior must not change: no signature changes, no files touched besides `BookingService`, no test edits. Run the suite and show me the diff.

- **Name the refactor.** "Extract" rather than "improve".
- **Bound the scope.** Which files, which method, what is off-limits.
- **State what must not change.** Behavior, signatures, tests, etc.

Scope discipline

We asked for one extract in `BookingService`. The diff came back like this:

```
$ git diff --stat
booking/BookingService.java | 21 ++++++-----
booking/RoomService.java    | 14 +++-----
domain/Booking.java         |  6 +-
domain/Slot.java             |  3 +-

```

- (From before: a big diff for a small ask, including files we never mentioned.)
- What's new: for a named refactor, the footprint is predictable before the diff exists. One file, a handful of lines.
- Glance at the stat, and reject or re-scope anything beyond the named refactor.

Checking the refactor

Verifying the refactor

- A green suite is necessary (but not sufficient, since the suite may not cover the boundary that moved).
- The bar: the suite is green, the pins hold, and the diff reads as behavior-neutral.
- The third needs some reading. One extract is a handful of lines. A repo-wide move can be hundreds or thousands.
 - One concern per pass. Behavior changes first, then conventions.

What hides in a "cleanup"

- Reordered checks. Changed defaults. A "simplified" condition that is not equivalent. And so on.
- Error types and messages swapped (someone's code could rely on those).
- "Dead" code removed that still ran in rare cases.

Is this still a refactoring?

We asked an agent to tidy `book()`.

`countFor` returns how many bookings the room has that day, and `feeFor` turns that count into a fee.

```
// before
Receipt book(User u, Room r, Slot s) {
    // checks ...
    int earlier = store.countFor(r, s.day());
    Booking b = store.add(new Booking(u, r, s));
    return new Receipt(b, feeFor(earlier));
}
```

```
// after
Receipt book(User u, Room r, Slot s) {
    // checks ...
    Booking b = store.add(new Booking(u, r, s));
    return new Receipt(b,
        feeFor(store.countFor(r, s.day())));
}
```

The suite is green. Did behavior change? Does it even matter?

The read moved after the write

- `countFor` now runs after `add`, so it counts the new booking too.
- The fee differs only where `feeFor` changes tier (e.g., the third booking for a day).
- Every test and every pin passed, because none booked at a tier boundary.
- Pins only cover the inputs they run, so we choose them like tests (classes and boundaries, from Week 2).
- Inlining a variable is safe when nothing in between changes what the expression returns. Here, `add` does.

```
// before
Receipt book(User u, Room r, Slot s) {
    // checks ...
    int earlier = store.countFor(r, s.day());
    Booking b = store.add(new Booking(u, r, s));
    return new Receipt(b, feeFor(earlier));
}
```

```
// after
Receipt book(User u, Room r, Slot s) {
    // checks ...
    Booking b = store.add(new Booking(u, r, s));
    return new Receipt(b,
        feeFor(store.countFor(r, s.day())));
}
```

Did it get better?

- "Tests pass" only says the code is no worse.
- Is the next likely change now a smaller diff?
 - Ask the agent which files that change would touch, before the refactor and after.
- Can we say what each extracted piece is **for**, in one sentence?
 - Try to write the sentence.
- Did the dependency arrows get fewer, or point in a better direction?
 - Have the agent draw the import graph, before and after.
- If the answer is "no" or "it just looks cleaner", the structure did not improve.

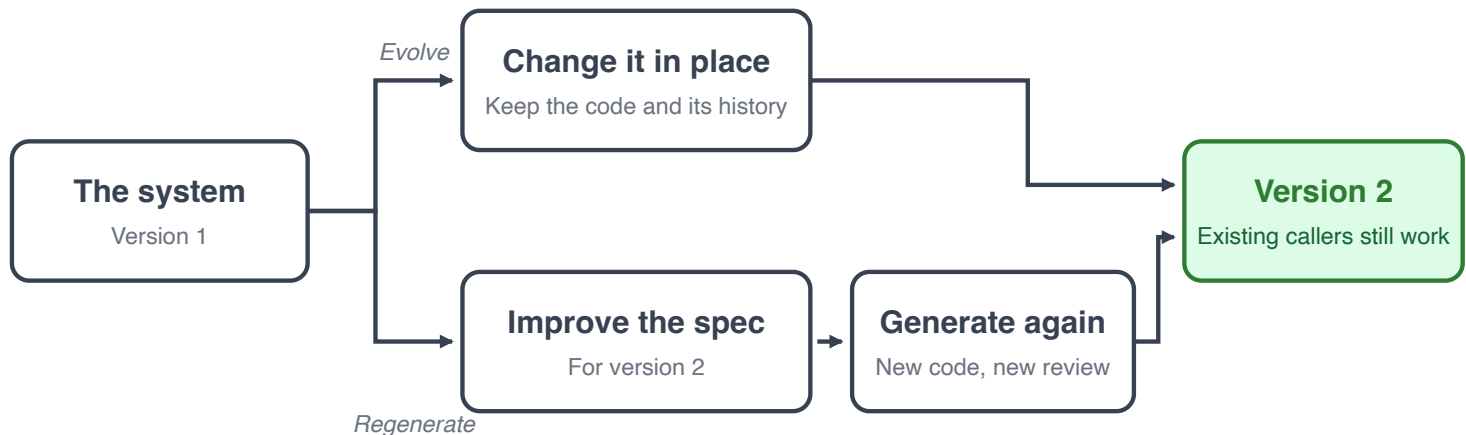
What is different about generated code?

- The fixes are the usual ones (call the existing helper, remove layers that do nothing, rename to match the repo).
- The next agent session reads this code and tends to copy what it finds. A duplicate or a drifted name left in place can spread.
- So each cleanup also makes the **next** generation fit the repo better.
- There is also a second option. We could generate it again. Thoughts?

Refactor or regenerate?

The option we didn't really have before

- For most of software history, code was too expensive to throw away. Rewrites were rare and risky, so refactoring was usually the only road.
- Now we can delete it, improve the **spec**, and regenerate, often in minutes.



When regenerating tends to win

- **Wrong structure.** Refactoring can fix the surface, but moving the boundaries is expensive.
- **A crisp spec.** It exists, or is cheap to write.
 - As a total aside: what do we think of specs written against the code (and not the other way round)?
- **Little worth keeping.** Few fixes since generation, no accumulated knowledge in the history.
- **A strong suite.** It can check whatever comes back.

When refactoring tends to win

- **Embedded knowledge.** Months of bug fixes live in the history, and regeneration may discard **all of it**.
- **The tests grew around this structure.** So did the harness, and your clients (à la Assignment 3).
- **Mostly right structure.** The boundaries are right and the surface is messy.
- **The checking is already done.** Every part you reviewed in Assignment 2 stays reviewed.

Regeneration is not free

- All of the code is new, so the review budget applies to all of it again.
 - Glance, let the agent search and report, spot-check, read closely where the risk is.
- The design is new too, so you have to work out again which decisions deserve a close review. (When you evolve, you only review what changed.)
- The bugs are new and unknown. The old ones may be known and have workarounds.
- Whatever comes back still has to work with the version 1 client, against code you probably have not checked yet.
- The harness only catches what it checks. Anything it does not check can change without you noticing.

Tracing a request through the code

Suppose version 2 of the reservation system says **some rooms are restricted to a group**.

One way to do this

1. Start from the contract diff and list each new rule.
2. For each rule, follow one request from its entry point, call by call.
 - At each call (a hop), note what is checked and what state changes. Stop at storage.
3. Where a new rule has no home in the trace, that is code that has to change.

Ask your agent to produce the trace, and spot-check it. (Or verify more closely along a path that is more difficult to change later.)

You can trace through code to find **design** gaps.

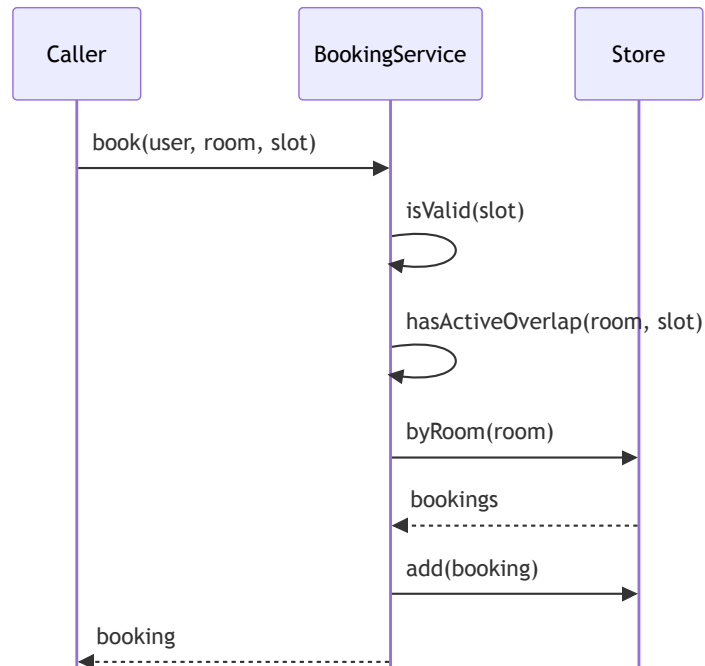
- A new field is data, a new meaning for cancel is an operation, and a rule with no home is structure, etc.

Trace: `book(user, room, slot)`

We asked an agent to trace one request.

```
book(user, room, slot)      BookingService:18
├─ isValid(slot)            :21
│   └─ throws InvalidSlot
├─ hasActiveOverlap(room, slot) :24
│   └─ throws SlotTaken
├─ store.byRoom(room)       :41
│   └─ reads bookings
└─ store.add(booking)       :27
    └─ writes the booking
```

Version 2 says some rooms are restricted to a group. No hop looks at who the user is. The rule has no home, and where it goes (`book()` , `Room` , or a policy object) is a structure decision.



Two roads for the same change

Evolve

- **Touch:** Room (the group), book () (the new check), the room list (hide what the user may not book), and whatever creates rooms.
- **Re-verify:** Those paths, and every existing caller of book () .
- **Read:** A diff.

Regenerate

- **Write first:** The version 2 spec, with judgment calls.
- **Touch:** Everything.*
- **Re-verify:** Everything.*
- **Read:** Glance, spot-check, read closely where the risk is. All over again.

* We can regenerate one part and evolve the rest, if that part has a clear boundary.

Which road?

It depends.

- **Test coverage.** What do your tests and harness pin? The list of checks is a first answer.
- **Code age.** How many fixes (yours or your agent's) have gone in since it was generated?
- **Spec quality.** Could someone rebuild the system from the contract and `CAPABILITY.md` alone? (That was the Assignment 2 bar.)
- **Reach.** Who depends on exact behavior? The version 1 client, the front end, your own harness.

Refactor or regenerate?

Pick a road for each, and explain why.

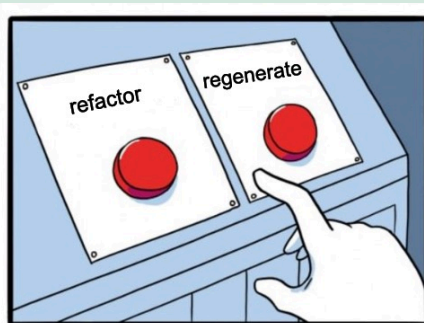
A. A 900-line generated parser, three weeks old, with a short and precise spec. No tests. Two other services depend on its exact error messages.

B. A five-year-old billing module. 300 green tests, forty edge-case fixes over time. One 800-line method holds every tax rule, and a new country needs to be supported next month.

C. An agent-built API layer from last month. Decent tests, working, but the conventions drifted from the rest of the repo.



Canvas →
Quizzes →
Lecture 9
In-Class Quiz



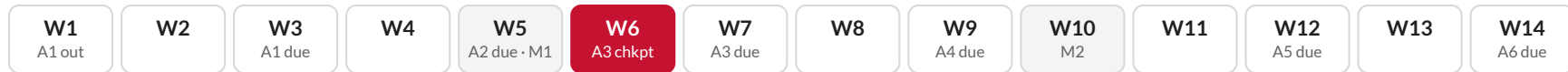
Assignment 3's checkpoint is Wednesday

- `AUDIT.md` at the root of your repository, by Wednesday night. Four parts:
 - What each new feature affects (a scope sentence and a short list of places).
 - Two design smells.
 - What makes version 1 easy or hard to keep.
 - Evolve or regenerate, and why.
- For Assignment 3, choose one road for the whole system.
 - Your transcripts and the defense should show that *you*, and not your agent, decided.
- At the deadline you add a few sentences on what building showed. The checkpoint text stays as written.
- Either road: the version 1 client must still work, and decisions get superseded (not rewritten).

Summary

- Refactoring changes structure and preserves externally observable behavior. If behavior changed, it was not a refactoring.
- Wear one hat at a time (restructure, or add behavior), and switch at a commit boundary.
- Characterization tests pin what the code **does**, oddities included, before we touch it. Generating them from the code is fine here.
- Direct an agent's refactor by naming it, bounding the scope, and stating what must not change. A green test suite is not enough, so check (or spot-check) the diff.
- Refactor or regenerate is a judgment call. Trace what the change touches, then decide (coverage, age, spec quality, reach).

What's next



- **Wednesday:** Design patterns and their tradeoffs, and the Open/Closed principle. Again, the A3 checkpoint is due that night.
- **Friday, Lab 6:** An API contract under pressure.
- **Next Wednesday:** A3 due.